

SERVICIOS DE FIREBASE

Firebase Authentication con Correo Gmail

Configuración paso a paso, desde la creación del proyecto en Firebase hasta el código Java funcionando con Credential Manager.

GRUPO 1

Tema: Gmail · UTEQ – Ingeniería en Software



Qué vamos a cubrir

01

Qué es Firebase Authentication

02

Cómo funciona el login con Google/Gmail

03

Crear el proyecto en Firebase Console

04

Conectar Android Studio con Firebase

05

Habilitar el proveedor Google + SHA-1

06

google-services.json y el Web Client ID

07

Código Java: login, credencial y sesión

08

Errores comunes al probar la demo

09

Ventajas y desventajas

Integrantes y responsabilidades

1

Darién Alberto Benites Pérez

Introducción: qué es Firebase Authentication y cómo funciona el login con Google/Gmail (flujo conceptual).

2

Erwin Daniel Bueno Troya

Configurar el proyecto en Firebase Console, conectarlo desde Android Studio, habilitar el proveedor Google, registrar el SHA-1 y obtener el Web Client ID.

3

Mario Sebastián Castro Coello

Código Java: layout, login/logout con Credential Manager, y demo en vivo con manejo de errores.

4

Elián Jossué Mendoza Villarroel

Código Java: layout, login/logout con Credential Manager, y demo en vivo con manejo de errores.

¿Qué es Firebase Authentication?

Es un servicio de Firebase (Google) que provee backend, SDKs listos y componentes de UI para autenticar usuarios en apps móviles y web, sin construir ni administrar un servidor de autenticación propio.

Múltiples proveedores

Correo/contraseña, Google, Facebook, Twitter, GitHub, teléfono, anónimo.

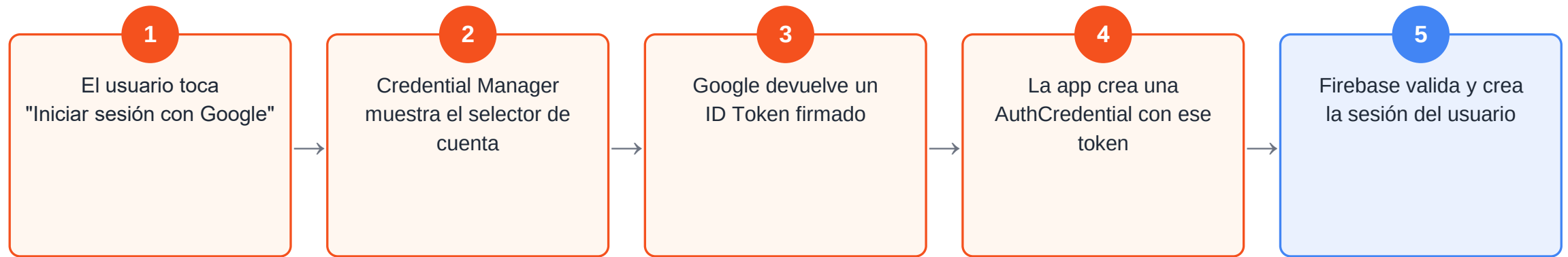
Tokens seguros

Maneja ID Tokens verificados por Google para crear la identidad del usuario.

Integración con Firebase

Se conecta directo con Firestore, Realtime DB y reglas de seguridad.

Cómo funciona el login con Gmail



Usamos la librería `androidx.credentials` (Credential Manager), la forma actual recomendada por Google — reemplaza a la antigua `GoogleSignInClient`.

Crear el proyecto en Firebase Console

1

Entrar a console.firebase.google.com con una cuenta Google

2

Clic en "Agregar proyecto" y ponerle un nombre (ej: FirebaseAuthGmailDemo)

3

Desactivar Google Analytics (no es necesario para esta demo)

4

Esperar a que Firebase termine de crear el proyecto

Conectar Android Studio con Firebase

1

En Android Studio: Tools → Firebase

2

En el panel derecho, expandir "Authentication"

3

Elegir "Authenticate using Google [Java]" (versión Java, no Kotlin)

4

Clic en "Connect to Firebase" → iniciar sesión con la cuenta de Google

5

Elegir el proyecto ya creado en Firebase Console y confirmar el applicationId

Este asistente hace automáticamente lo que antes se hacía a mano: registrar la app y descargar google-services.json.

Agregar el SDK de Firebase Authentication

- 1 En el mismo panel de Firebase Assistant, clic en "Add the Firebase Authentication SDK to your app"
- 2 Android Studio modifica automáticamente el build.gradle raíz y el de app
- 3 Aparece un diálogo "Accept Changes" → aceptar
- 4 Esperar el sync de Gradle hasta que termine sin errores

build.gradle (app) — dependencias agregadas automáticamente

```
dependencies {  
    implementation platform('com.google.firebase:firebase-bom:33.13.0')  
    implementation 'com.google.firebase:firebase-auth'  
    implementation 'androidx.credentials:credentials:1.3.0'  
    implementation 'androidx.credentials:credentials-play-services-auth:1.3.0'  
    implementation 'com.google.android.libraries.identity.googleid:googleid:1.1.1'
```


Habilitar el proveedor Google

1

En Firebase Console: Authentication → pestaña "Método de acceso"

2

Clic en "Google" dentro de Proveedores de acceso

3

Activar el switch "Habilitar"

4

Elegir un "Correo electrónico de asistencia del proyecto" (obligatorio)

5

Clic en "Guardar"

Error común: si no seleccionas el correo de asistencia, Firebase no deja guardar el proveedor habilitado.

Obtener y registrar el SHA-1

En la Terminal de Android Studio, dentro de la carpeta del proyecto:

```
./gradlew signingReport

// En Windows (sin ./ al inicio):
gradlew signingReport
```

Busca el bloque "Variant: debug" y copia el valor de "SHA1:".

Registrar el SHA-1 en Firebase

1. Firebase Console →  Configuración del proyecto
2. Pestaña "Tus apps" → selecciona tu app Android
3. Sección "Huellas digitales del certificado SHA"
4. Clic en "Agregar huella digital"
5. Pega el SHA1 copiado → Guardar

google-services.json y el Web Client ID

1 Firebase Console → Configuración del proyecto → Tus apps → descargar google-services.json

2 Reemplazar el archivo dentro de la carpeta app/ del proyecto

3 Abrir el archivo y ubicar el arreglo "oauth_client"

4 Copiar el client_id del objeto que tenga "client_type": 3

```
"oauth_client": [  
  { "client_id": "...apps.googleusercontent.com", "client_type": 1 },  
  { "client_id": "...apps.googleusercontent.com", "client_type": 3 } // <- este  
]
```

Si "oauth_client" aparece vacío, vuelve a descargar el archivo: se genera solo después de habilitar Google y guardar el SHA-1.

Agregar el Web Client ID en strings.xml

```
<!-- res/values/strings.xml -->
<resources>
    <string name="app_name">FirebaseAuthGmailDemo</string>
    <string name="default_web_client_id">
        367904972229-p2prn8fqle9t51q8f4ds3nj57j0r5prh.apps.googleusercontent.com
    </string>
</resources>
```

Este valor identifica tu proyecto de Firebase ante Google al pedir el ID Token — sin él, Credential Manager no sabe a qué backend autenticar.

Layout: activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="No has iniciado sesión"
        android:textSize="16sp" />

    <Button android:id="@+id/btnGoogleSignIn"
        android:text="Iniciar sesión con Google" ... />

    <Button android:id="@+id/btnSignOut"
        android:text="Cerrar sesión"
        android:visibility="gone" ... />

</LinearLayout>
```

MainActivity.java — iniciar sesión

```
auth = FirebaseAuth.getInstance();
credentialManager = CredentialManager.create(this);

btnSignIn.setOnClickListener(v -> signIn());
btnSignOut.setOnClickListener(v -> signOut());

private void signIn() {
    GetGoogleIdOption googleIdOption = new GetGoogleIdOption.Builder()
        .setFilterByAuthorizedAccounts(false)
        .setServerClientId(getString(R.string.default_web_client_id))
        .build();

    GetCredentialRequest request = new GetCredentialRequest.Builder()
        .addCredentialOption(googleIdOption)
        .build();

    credentialManager.getCredentialAsync(
        this, request, null, executor,
        new CredentialManagerCallback<>() {
            public void onResult(GetCredentialResponse r) {
                handleCredential(r.getCredential());
            }
            public void onError(GetCredentialException e) { ... }
        });
}
```

MainActivity.java — validar credencial con Firebase

```
private void handleCredential(Credential credential) {  
    if (credential instanceof GoogleIdTokenCredential) {  
        GoogleIdTokenCredential googleCred =  
            (GoogleIdTokenCredential) credential;  
        firebaseAuthWithGoogle(googleCred.getIdToken());  
    }  
}  
  
private void firebaseAuthWithGoogle(String idToken) {  
    AuthCredential credential =  
        GoogleAuthProvider.getCredential(idToken, null);  
  
    auth.signInWithCredential(credential)  
        .addOnCompleteListener(this, task -> {  
            if (task.isSuccessful()) {  
                updateUI(auth.getCurrentUser());  
            } else {  
                Toast.makeText(this, "Error de autenticación",  
                    Toast.LENGTH_SHORT).show();  
                updateUI(null);  
            }  
        });  
}
```

Cerrar sesión y reflejar el estado en la UI

```
private void signOut() {
    auth.signOut();
    updateUI(null);
}

private void updateUI(FirebaseUser user) {
    if (user != null) {
        tvStatus.setText("Sesión iniciada como:\n" +
user.getEmail());
        btnSignIn.setVisibility(View.GONE);
        btnSignOut.setVisibility(View.VISIBLE);
    } else {
        tvStatus.setText("No has iniciado sesión");
        btnSignIn.setVisibility(View.VISIBLE);
        btnSignOut.setVisibility(View.GONE);
    }
}
```

onStart()

Al volver a abrir la app llamamos `updateUI(auth.getCurrentUser())` para reflejar si ya había una sesión activa, sin pedir login otra vez.

Errores comunes al probar la demo



"No credentials available"

El emulador no tiene ninguna cuenta de Google agregada en Ajustes → Cuentas, o el emulador no incluye Play Store.



"Something went wrong" al agregar cuenta

Casi siempre es la fecha/hora del emulador desincronizada. Revisar Ajustes → Sistema → Fecha y hora automáticas.



oauth_client vacío en el JSON

El archivo se descargó antes de habilitar Google o agregar el SHA-1. Hay que volver a descargarlo.



No aparece default_web_client_id

El asistente de Firebase no siempre lo genera; se agrega a mano en strings.xml con el client_id de client_type 3.

Ventajas



Registro instantáneo

El usuario no crea ni recuerda una contraseña nueva; usa la cuenta que ya tiene.



Seguridad delegada a Google

2FA, detección de dispositivos y protección contra phishing las maneja Google.



Menos fricción, más conversión

Un solo toque para iniciar sesión reduce el abandono en el registro.



Datos verificados

El correo llega ya verificado por Google, sin flujo extra de verificación.

Desventajas



Dependencia de Google Play Services

No funciona en dispositivos sin GMS (algunos Huawei, ROMs personalizadas).



El usuario debe tener Gmail

Excluye a quien no usa o no quiere usar una cuenta de Google.



Menos control de marca

El flujo de selección de cuenta lo maneja la UI de Google, no la tuya.



Riesgo de cuenta única

Si la cuenta de Google se compromete, también se compromete el acceso a la app.

CONCLUSIÓN

Login rápido, seguro y sin fricción

Ya funcionando de extremo a extremo: proyecto en Firebase, proveedor Google habilitado, SHA-1 registrado y la app Java autenticando con Credential Manager.

Demo en vivo a continuación →

